TP 4 - TDD, Performance & Sécurité

Test-Driven Development · Locust · Bandit · OWASP

Objectif du TP

Ce TP final applique trois stratégies avancées vues en CM : (1) implémenter une nouvelle fonctionnalité de ShopFlow en cycle TDD strict Red→Green→Refactor, (2) mesurer les performances de l'API sous charge avec Locust, (3) analyser et corriger les vulnérabilités de sécurité détectées par Bandit.

Stack	pytest (TDD) · Locust · Bandit · FastAPI · SQLite
Prérequis	TP0 à TP3 complétés, API ShopFlow démarrée

Partie 1 - TDD : Système de Coupons ShopFlow

Dans cette partie, vous allez implémenter la fonctionnalité **validation avancée des coupons** en suivant strictement le cycle TDD : **RED → GREEN → REFACTOR**. Le code métier n'existe pas encore et vous écrirez les tests en premier.

1.1 Rappel du cycle TDD

RED

- Écrire un test qui ÉCHOUE
- ImportError ou AssertionError attendu
- La fonction n'existe pas encore

GREEN

- Écrire le MINIMUM de code
- Faire passer le test, rien de plus
- Pas d'optimisation ici

REFACTOR

- Améliorer le code proprement
- Sans casser les tests existants
- Clarté, DRY, robustesse

1.2 Contexte -Nouvelle fonctionnalité : Validation avancée

Le client ShopFlow demande des règles de validation plus strictes pour les coupons :

Spécifications de la nouvelle fonctionnalité

- valider_coupon(coupon, panier_total) → retourne True/False selon les règles ci-dessous
- Règle 1 : Le coupon doit être actif (actif=True)
- Règle 2 : La réduction doit être dans l'intervalle]0, 100]
- Règle 3 : Le coupon ne peut être appliqué que si panier_total >= 10.0 (montant minimum)
- Règle 4 : Un coupon avec réduction=100 (gratuit) n'est valide que si panier_total >= 50.0
- Règle 5 : Lever ValueError avec message explicite pour chaque règle violée

1.3 Étape RED — Écrire les tests en premier

Créez le fichier `tests/unit/test_coupon_tdd.py`. **Ne touchez pas encore à pricing.py** ,les tests doivent échouer :

tests/unit/test_coupon_tdd.py - Phase RED

```python
# tests/unit/test_coupon_tdd.py - TDD : écrire les tests AVANT le code
import pytest
# L'import suivant va échouer (ImportError) — c'est normal en RED !
from app.services.pricing import valider_coupon
from app.models import Coupon

class TestValiderCoupon:

    def test_coupon_actif_valide(self):
        """Coupon actif + montant suffisant → True."""
        c = Coupon(code='PROMO20', reduction=20.0, actif=True)
        assert valider_coupon(c, panier_total=50.0) is True

    def test_coupon_inactif_leve_exception(self):
        c = Coupon(code='OLD', reduction=10.0, actif=False)
        with pytest.raises(ValueError, match='inactif'):
            valider_coupon(c, panier_total=50.0)

    def test_montant_minimum_non_atteint(self):
        """Panier < 10€ → ValueError."""
        c = Coupon(code='PROMO10', reduction=10.0, actif=True)
```

```python
        with pytest.raises(ValueError, match='minimum'):
            valider_coupon(c, panier_total=5.0)

    def test_coupon_gratuit_panier_insuffisant(self):
        """reduction=100 mais panier < 50€ → ValueError."""
        c = Coupon(code='FREE', reduction=100.0, actif=True)
        with pytest.raises(ValueError, match='50'):
            valider_coupon(c, panier_total=30.0)

    def test_coupon_gratuit_panier_suffisant(self):
        """reduction=100 et panier >= 50€ → True."""
        c = Coupon(code='FREE100', reduction=100.0, actif=True)
        assert valider_coupon(c, panier_total=50.0) is True

    def test_reduction_invalide_leve_exception(self):
        c = Coupon(code='BAD', reduction=0.0, actif=True)
        with pytest.raises(ValueError, match='réduction'):
            valider_coupon(c, panier_total=100.0)

    @pytest.mark.parametrize('total,attendu', [
        (10.0,  True),   # exactement le minimum
        (9.99,  False),  # juste en dessous du minimum
        (100.0, True),   # largement suffisant
    ])
    def test_seuil_montant_minimum(self, total, attendu):
        c = Coupon(code='T', reduction=10.0, actif=True)
        if attendu:
            assert valider_coupon(c, panier_total=total) is True
        else:
            with pytest.raises(ValueError):
                valider_coupon(c, panier_total=total)
```

```
Terminal — Phase RED
# Lancer les tests — ils doivent TOUS échouer (ImportError)
pytest tests/unit/test_coupon_tdd.py -v

# Sortie attendue en RED :
# ERROR tests/unit/test_coupon_tdd.py - ImportError:
# cannot import name 'valider_coupon' from 'app.services.pricing'
# ← C'est NORMAL et VOULU en phase RED
```

1.4 Étape GREEN - Implémenter le minimum

Maintenant seulement, ajoutez `valider_coupon()` dans `app/services/pricing.py`. Écrivez le **minimum de code pour faire passer les tests** (pas d'optimisation) :

```
app/services/pricing.py — squelette à compléter
# À ajouter dans app/services/pricing.py
# (après les fonctions existantes)

MONTANT_MINIMUM_COUPON = 10.0   # Seuil minimum pour utiliser un coupon
MONTANT_MINIMUM_GRATUIT = 50.0  # Seuil pour coupon 100% (gratuit)

def valider_coupon(coupon: Coupon, panier_total: float) -> bool:
    """
    Valide qu'un coupon peut être appliqué sur un panier.
    Lève ValueError avec message explicite si une règle est violée.
    """
    # Règle 1 : coupon actif
    # Règle 2 : réduction valide
    # Règle 3 : montant minimum
    # Règle 4 : coupon gratuit → seuil plus élevé
    # À COMPLÉTER — implémentez les 4 règles
    ...
```

```
Terminal — Phase GREEN
# Lancer les tests après GREEN
pytest tests/unit/test_coupon_tdd.py -v

# Sortie attendue en GREEN :
# test_coupon_actif_valide          PASSED
# test_coupon_inactif_leve_exception PASSED
# test_montant_minimum_non_atteint   PASSED
# test_coupon_gratuit_panier_insuf   PASSED
# test_coupon_gratuit_panier_suf     PASSED
# test_reduction_invalide            PASSED
# test_seuil_montant_minimum[...]    PASSED (×3)
# 9 passed in 0.05s
```

1.5 Étape REFACTOR - Améliorer sans casser

Maintenant que les tests passent, améliorez l'implémentation pour la rendre plus robuste et lisible. **Les tests doivent continuer à passer après chaque modification.**

Points de refactoring à appliquer

- Extraire les constantes MONTANT_MINIMUM_COUPON et MONTANT_MINIMUM_GRATUIT dans un fichier app/config.py
- Ajouter des messages d'erreur explicites avec le montant exact du seuil non atteint
- Ajouter un docstring complet avec les paramètres, retour et exceptions documentés
- Écrire 2 tests supplémentaires pour les cas limites (panier_total=0, panier_total négatif)
- Vérifier que pytest --cov=app coverage est ≥ 80% après l'ajout de valider_coupon

```
Terminal — vérification post-REFACTOR
# Après REFACTOR — vérification complète
pytest tests/unit/ -v --cov=app --cov-report=term-missing

# Tous les tests existants + nouveaux doivent passer
# Coverage doit rester ≥ 80%
```

Q1 :TDD Coupons ShopFlow

1.1. Lancez pytest test_coupon_tdd.py en phase RED (avant d'écrire valider_coupon). Copiez la sortie d'erreur montrant l'ImportError. Expliquez pourquoi cet échec est VOULU en TDD.

1.2. Implémentez valider_coupon() dans pricing.py (phase GREEN). Copiez la sortie de pytest montrant les 9 tests PASSED.

1.3. Appliquez les points de refactoring listés. Montrez le code refactorisé de valider_coupon() avec les messages d'erreur améliorés.

1.4. Comparez l'approche TDD (test d'abord) avec l'approche classique (code d'abord, tests ensuite, comme en TP1). Citez 2 avantages concrets que TDD vous a apportés dans cet exercice.

Partie 2 - Tests de Performance avec Locust

Dans cette partie, vous allez utiliser Locust pour simuler une charge utilisateur sur ShopFlow et identifier les endpoints les plus lents ou les plus fragiles sous pression.

2.1 Lancer l'API ShopFlow

Terminal 1 — Démarrage API

```
# Terminal 1 — Démarrer l'API ShopFlow
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

# Vérifier que l'API répond
curl http://localhost:8000/health
# Attendu : {"status": "ok", ...}

# Pré-charger quelques produits de test
for i in $(seq 1 20); do
  curl -s -X POST http://localhost:8000/products/ \
    -H 'Content-Type: application/json' \
    -d "{\"name\": \"Produit $i\", \"price\": $((i*10)).99, \"stock\": 100}" > /dev/null
done
echo "20 produits créés"
```

2.2 Scénarios Locust

Complétez le fichier `tests/perf/locustfile.py` avec les scénarios suivants. La structure de base est déjà fournie dans le projet ShopFlow, ajoutez les tâches manquantes :

tests/perf/locustfile.py — à compléter

```
# tests/perf/locustfile.py — à compléter
from locust import HttpUser, task, between
import random

PRODUCT_IDS = list(range(1, 21))   # IDs des 20 produits créés

class ShopFlowUser(HttpUser):
    wait_time = between(0.5, 2.0)

    def on_start(self):
        self.user_id = random.randint(10000, 99999)

    @task(5)
    def browse_products(self):
        """Scénario le plus fréquent : lister les produits."""
        self.client.get('/products/', name='/products/ [list]')

    @task(3)
    def get_product_detail(self):
        """Consulter un produit au hasard."""
        pid = random.choice(PRODUCT_IDS)
        self.client.get(f'/products/{pid}', name='/products/{id}')

    @task(2)
    def add_to_cart(self):
        # À COMPLÉTER : POST /cart/?user_id=self.user_id
        # body : {"product_id": random.choice(PRODUCT_IDS), "quantity": 1}
        ...

    @task(1)
    def place_order(self):
        # À COMPLÉTER :
        # 1. ajouter un produit au panier (POST /cart/)
        # 2. passer la commande (POST /orders/)
        # Gérer le cas 400 (panier vide) comme succès acceptable
```

```
        ...

    @task(1)
    def health_check(self):
        self.client.get('/health', name='/health [smoke]')
```

2.3 Lancer Locust en mode interactif

```
Terminal 2 — Locust UI
# Terminal 2 — Lancer Locust UI
locust -f tests/perf/locustfile.py --host=http://localhost:8000

# Ouvrir : http://localhost:8089
# Configurer :
#   Number of users : 50
#   Spawn rate      : 5 users/sec
#   Host            : http://localhost:8000
# Lancer pendant 60 secondes
```

```
Mode headless avec rapport
# Mode headless (sans UI) — pour CI/CD
locust -f tests/perf/locustfile.py \
    --host=http://localhost:8000 \
    --headless \
    -u 50 -r 5 \
    --run-time 60s \
    --html=tests/perf/rapport_locust.html \
    --csv=tests/perf/resultats

# Vérifier les seuils après le test
python3 -c "
import csv
with open('tests/perf/resultats_stats.csv') as f:
    reader = csv.DictReader(f)
    for row in reader:
        if row['Name'] != 'Aggregated':
            p95 = float(row.get('95%', 0))
            print(f\"{row['Name']:40} p95={p95}ms\")
            if p95 > 500:
                print(f'SEUIL DÉPASSÉ (> 500ms)')
"
```

2.4 Seuils de performance à respecter

Endpoint	p95 cible	p99 cible	Priorité
`GET /health`	**< 50ms**	< 100ms	Critique
`GET /products/ [list]`	**< 200ms**	< 500ms	Haute
`GET /products/{id}`	**< 150ms**	< 300ms	Haute
`POST /cart/ [add]`	**< 300ms**	< 700ms	Moyenne
`POST /orders/ [create]`	**< 500ms**	< 1000ms	Moyenne

Q2 : Tests de Performance Locust

2.1. Complétez add_to_cart() et place_order() dans locustfile.py. Lancez Locust en mode interactif (50 users, 60s). Copiez le tableau de statistiques affiché dans Locust UI (colonnes : Name, Requests, Failures, Median, p95, p99).

2.2. Identifiez l'endpoint le plus lent (p95 le plus élevé). Est-il dans les seuils du tableau ci-dessus ? Si non, proposez une optimisation concrète pour ShopFlow.

2.3. Lancez Locust en mode headless avec rapport HTML. Ouvrez tests/perf/rapport_locust.html. Copiez ou décrivez les 3 graphiques présents dans le rapport.

2.4. Augmentez la charge à 200 users avec spawn rate 20. Observez le comportement de l'API sous forte charge. Y a-t-il des erreurs (Failures > 0) ? Expliquez ce qui se passe.

2.5. Expliquez la différence entre un test de charge (load test), un test de stress (stress test) et un test d'endurance (soak test). Dans quel scénario Locust utiliseriez-vous chacun pour ShopFlow ?

Partie 3 : Analyse de Sécurité avec Bandit

Dans cette partie, vous allez analyser ShopFlow avec Bandit, lire et interpréter le rapport de vulnérabilités, et corriger au moins deux problèmes identifiés.

3.1 Lancer l'analyse Bandit

```
Terminal — Bandit
# Analyse complète avec rapport JSON
bandit -r app/ \
    -f json \
    -o bandit-report.json \
    -v

# Rapport lisible dans le terminal
bandit -r app/ -ll   # -ll = seulement MEDIUM et HIGH

# Rapport HTML
bandit -r app/ -f html -o bandit-report.html

# Compter les vulnérabilités par sévérité
python3 -c "
import json
data = json.load(open('bandit-report.json'))
from collections import Counter
sev = Counter(r['issue_severity'] for r in data['results'])
print('HIGH   :', sev.get('HIGH', 0))
print('MEDIUM :', sev.get('MEDIUM', 0))
print('LOW    :', sev.get('LOW', 0))
"
```

3.2 Comprendre le rapport Bandit

Voici un exemple de vulnérabilité typique que Bandit peut détecter dans ShopFlow :

```
Exemple de résultat Bandit
# Exemple de résultat bandit-report.json
{
  "filename": "app/cache.py",
  "test_id": "B105",
  "test_name": "hardcoded_password_string",
  "issue_text": "Possible hardcoded password: 'admin'",
  "issue_severity": "MEDIUM",
  "issue_confidence": "MEDIUM",
  "line_number": 12,
  "more_info":
"https://bandit.readthedocs.io/en/latest/plugins/b105_hardcoded_password_string.html"
}
```

ID	Nom	Description	Correction
B105	**hardcoded_password**	Mot de passe codé en dur dans le code source	Utiliser os.getenv('PASSWORD')
B108	**probable_hardcoded_tmp**	Chemin temporaire /tmp/ codé en dur	Utiliser tempfile.mkdtemp()
B301	**pickle**	Désérialisation pickle (RCE possible)	Utiliser json.loads() à la place
B320	**xml_bad_element_tree**	XML External Entity (XXE)	Utiliser defusedxml

B501	**request_with_no_cert_verify**	SSL verify=False	Toujours verify=True en production
B506	**yaml_load**	yaml.load() (RCE via YAML)	Utiliser yaml.safe_load()

3.3 Exercice de correction

Introduisez volontairement 2 vulnérabilités dans le code ShopFlow, puis corrigez-les :

```
Vulnérabilités à introduire
# Vulnérabilité 1 à introduire dans app/cache.py
# (hardcoded password — B105)
REDIS_PASSWORD = 'admin123'   # ← NE JAMAIS FAIRE EN VRAI

# Vulnérabilité 2 à introduire dans app/main.py
# (désactivation SSL — B501) — dans une hypothétique fonction d'appel externe
import requests
response = requests.get('https://api.externe.com', verify=False)  # ← DANGEREUX
```

```
Corrections attendues
# Après correction — forme attendue

# Correction vulnérabilité 1 — utiliser os.getenv
REDIS_PASSWORD = os.getenv('REDIS_PASSWORD', '')  # bonne pratique

# Correction vulnérabilité 2 — toujours verify=True
response = requests.get('https://api.externe.com', verify=True)  # ok
# Ou mieux : utiliser un certificat CA personnalisé
response = requests.get('https://api.externe.com', verify='/path/to/ca.crt')
```

Q3 - Analyse Sécurité Bandit

3.1. Lancez bandit -r app/ -ll. Copiez la sortie complète (ou le résumé : nombre de HIGH, MEDIUM, LOW). Y a-t-il des vulnérabilités HIGH dans ShopFlow par défaut ?

3.2. Introduisez les 2 vulnérabilités décrites en 3.3. Relancez Bandit. Copiez la sortie montrant B105 et B501 détectés.

3.3. Corrigez les 2 vulnérabilités. Relancez Bandit. Vérifiez que B105 et B501 ont disparu du rapport. Copiez le diff git show ou le code avant/après.

3.4. Expliquez pourquoi verify=False dans requests.get() est une vulnérabilité de sécurité. Quel type d'attaque rend-il possible ?